

AWS CloudShell Runbook

Provisioning Infra + Launching MLflow for the End-to-End MLOps Lab

Covers only the AWS CloudShell / console steps. Application code (training scripts, API, Dockerfile, Terraform files, CI/CD workflow) lives in the GitHub repository — clone it as shown in Step 3.

Prerequisites

- An AWS account with console access, region **ap-south-1** (Mumbai).
- The project repository cloned/available on branch **mlops-lab-e2e** (replace the repo URL in Step 3 with your own).
- Permissions to create EC2, S3, ECR, and IAM resources.

Region matters

Every step below assumes region **ap-south-1**. EC2 key pairs and most resources are region-scoped — creating a key pair in one region and deploying in another is a common source of failures.

Step 1 — Create an EC2 Key Pair

In the AWS Console (top-right region selector set to **ap-south-1**): **EC2** → **Key Pairs** → **Create key pair**.

- Name: **mlops-lab-key**
- Type: RSA, Format: .pem
- Click **Create** — the private key downloads automatically.

Simpler alternative

You can skip the console entirely and create the key pair directly from CloudShell in Step 4 below — this also saves the .pem file straight into CloudShell so there's no upload/download step at all.

Step 2 — Open AWS CloudShell

Click the CloudShell icon (>_) in the top navigation bar of the AWS Console. Wait ~30 seconds for it to initialize. Confirm the region shown in CloudShell matches **ap-south-1** — if not, switch the console region first and reopen CloudShell.

Step 3 — Install Terraform in CloudShell

```
curl -O https://releases.hashicorp.com/terraform/1.9.0/terraform_1.9.0_linux_amd64.zip
unzip terraform_1.9.0_linux_amd64.zip
sudo mv terraform /usr/local/bin/
terraform -version
```

Step 4 — Clone the Repository

Replace the URL below with your own repository and branch.

```
git clone -b mlops-lab-e2e https://github.com/<your-org>/<your-repo>.git
cd <your-repo>/mlops-lab/infra
ls
# Expect to see: main.tf  variables.tf  outputs.tf
```

If the repo is private

HTTPS clone will prompt for a username/password — GitHub no longer accepts account passwords here. Either generate a Personal Access Token (GitHub → Settings → Developer settings → Personal access tokens) and use it as the password, or set up SSH keys in CloudShell (ssh-keygen, then add the public key under GitHub → Settings → SSH and GPG keys) and clone using the git@github.com:... URL instead.

Step 5 — Create the EC2 Key Pair via CLI (recommended)

This creates the key pair directly in the correct region and saves the private key inside CloudShell — no upload/download needed.

```
aws ec2 create-key-pair \
  --region ap-south-1 \
  --key-name mlops-lab-key \
  --query 'KeyMaterial' \
  --output text > ~/mlops-lab-key.pem

chmod 400 ~/mlops-lab-key.pem
```

Skip this step only if you already created the key pair via the console in Step 1 in the **ap-south-1** region.

Step 6 — Provision Infrastructure with Terraform

```
terraform init

terraform apply \
  -var="key_pair_name=mlops-lab-key" \
  -var="my_ip_cidr=$(curl -s ifconfig.me)/32"

# review the plan, then type: yes
```

This creates 8 resources: an S3 bucket (versioned, used for DVC and MLflow artifacts), an ECR repository, an IAM role + policy + instance profile (S3 read/write, ECR pull, CloudWatch), a security group (SSH restricted to your IP; ports 5000 and 8000 open to 0.0.0.0/0 for the lab), and the EC2 instance itself.

Step 7 — Capture the Terraform Outputs

```
terraform output
```

Output	Description
ec2_public_ip	Public IP of the MLOps host — used for SSH, MLflow UI, and the API.
ecr_repository_url	ECR repo URL — used later to push/pull the serving image.
s3_bucket	S3 bucket used as the MLflow artifact store.
mlflow_url	http://<ec2_public_ip>:5000 — convenience output, same as building it yourself.

Keep this terminal/tab open — you'll need **ec2_public_ip** for every remaining step.

Step 8 — SSH into the EC2 Host

```
ssh -i ~/mlops-lab-key.pem ubuntu@<ec2_public_ip>
```

Type **yes** if prompted to confirm the host fingerprint.

Step 9 — Install Docker on the EC2 Host

Known issue

The Terraform boot script's automated install can fail because 'docker-compose-plugin' is not a valid package name on Ubuntu 22.04, which aborts the entire apt-get command before Docker installs. If 'docker --version' fails below, this is why — the fix is to install Docker manually, once, as shown here.

```
sudo apt-get update -y
sudo apt-get install -y docker.io
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker ubuntu

# log out and back in for the group change to apply
exit
```

```
ssh -i ~/mlops-lab-key.pem ubuntu@<ec2_public_ip>
docker ps
```

"docker ps" should return an empty table with headers, no permission error. If it still says permission denied, prefix Docker commands with sudo for the rest of this session.

Step 10 — Launch the MLflow Tracking Server

```
mkdir -p /opt/mlflow

sudo docker run -d --restart always --name mlflow \
  -p 5000:5000 \
  -v /opt/mlflow:/mlflow \
  -e AWS_DEFAULT_REGION=ap-south-1 \
  python:3.11-slim bash -c "pip install mlflow boto3 && \
    mlflow server --backend-store-uri sqlite:///mlflow/mlflow.db \
    --default-artifact-root s3://<s3_bucket>/mlflow-artifacts \
    --host 0.0.0.0 --port 5000 \
    --allowed-hosts '*'"
```

Why --allowed-hosts is required

MLflow 3.x ships with host-header validation enabled by default and only allows 'localhost' out of the box, even when bound to 0.0.0.0. Without this flag, opening the UI by public IP returns "Invalid Host header - possible DNS rebinding attack detected." Passing --allowed-hosts '*' is acceptable for a short-lived lab; in a real deployment, list the specific hostname/IP instead of a wildcard.

The container installs mlflow/boto3 on first start, which takes roughly 30-40 seconds. Wait, then check:

```
sleep 40
sudo docker logs mlflow --tail 15
```

Look for Application startup complete in the log output — that confirms MLflow is ready.

Step 11 — Open the MLflow UI

In a browser, go to:

```
http://<ec2_public_ip>:5000
```

This loads the MLflow UI, where experiments, runs, and the model registry are all visible and browsable.

If the page won't load

1) Confirm the container is actually up: `sudo docker ps -a` should show STATUS "Up", not "Exited".
2) Test locally first with `curl http://localhost:5000` from inside the SSH session — if that works but the browser doesn't, test from AWS CloudShell too (`curl -v http://:5000`) to rule out a local network/firewall issue on your own machine rather than an AWS-side problem. 3) Double-check the EC2 instance's security group has an inbound rule for TCP port 5000 from 0.0.0.0/0 (EC2 → Security Groups → inbound rules tab).

Step 12 — Model Registry: Stages vs. Aliases

Newer MLflow versions (3.x) deprecate the classic Staging/Production "Stage" dropdown in favor of **Aliases**. After registering a model version, check which system your MLflow build uses:

- Go to **Model registry** (left sidebar) → your model name → the version.
- If you see a **Stage** dropdown, use the classic flow: select **Production**.
- If you see **Aliases: Add** instead (Stage shows "deprecated"), click **Add**, type `production`, and save.

This changes the model URI

Classic stages use `models:/<model-name>/Production` (slash). Aliases use `models:/<model-name>@production` (at-sign, lowercase by convention). Whichever your MLflow build uses, make sure the `MODEL_URI` environment variable passed to the serving container (and the default fallback in the API code) matches exactly.

Teardown — Stop Billing When Done

From CloudShell, in the same folder where you ran terraform apply:

```
cd ~/<your-repo>/mlops-lab/infra

terraform destroy \
  -var="key_pair_name=mlops-lab-key" \
  -var="my_ip_cidr=$(curl -s ifconfig.me)/32"

# review the plan, then type: yes
```

S3 bucket contents

The bucket is created with `force_destroy = true`, so terraform destroy removes it along with everything inside — including MLflow run artifacts and the DVC data store. Re-running the training script after the next terraform apply recreates a fresh model in seconds, so this is generally fine for a lab environment.

Confirm everything is gone:

```
terraform show
# should return empty state
```

Appendix — Full Command Sequence (Quick Reference)

```

# 1. Install Terraform
curl -O https://releases.hashicorp.com/terraform/1.9.0/terraform_1.9.0_linux_amd64.zip
unzip terraform_1.9.0_linux_amd64.zip && sudo mv terraform /usr/local/bin/

# 2. Clone repo
git clone -b mlops-lab-e2e https://github.com/<org>/<repo>.git
cd <repo>/mlops-lab/infra

# 3. Create key pair
aws ec2 create-key-pair --region ap-south-1 --key-name mlops-lab-key \
  --query 'KeyMaterial' --output text > ~/mlops-lab-key.pem
chmod 400 ~/mlops-lab-key.pem

# 4. Provision infra
terraform init
terraform apply -var="key_pair_name=mlops-lab-key" \
  -var="my_ip_cidr=$(curl -s ifconfig.me)/32"
terraform output

# 5. SSH in and install Docker
ssh -i ~/mlops-lab-key.pem ubuntu@<ec2_public_ip>
sudo apt-get update -y && sudo apt-get install -y docker.io
sudo systemctl enable docker && sudo systemctl start docker
sudo usermod -aG docker ubuntu
exit
ssh -i ~/mlops-lab-key.pem ubuntu@<ec2_public_ip>

# 6. Launch MLflow
mkdir -p /opt/mlflow
sudo docker run -d --restart always --name mlflow \
  -p 5000:5000 -v /opt/mlflow:/mlflow \
  -e AWS_DEFAULT_REGION=ap-south-1 \
  python:3.11-slim bash -c "pip install mlflow boto3 && \
    mlflow server --backend-store-uri sqlite:///mlflow/mlflow.db \
    --default-artifact-root s3://<s3_bucket>/mlflow-artifacts \
    --host 0.0.0.0 --port 5000 --allowed-hosts '*'"
sleep 40 && sudo docker logs mlflow --tail 15

# 7. Open MLflow UI
# http://<ec2_public_ip>:5000

# 8. Teardown when finished
cd ~/<repo>/mlops-lab/infra
terraform destroy -var="key_pair_name=mlops-lab-key" \
  -var="my_ip_cidr=$(curl -s ifconfig.me)/32"

```

Application code (training, API, Dockerfile, CI/CD workflow) is not reproduced here — refer to the GitHub repository, branch mlops-lab-e2e, folder mlops-lab/.